

Table of Contents

1.	How do I get my code on Github	1
	Option 1.....	2
	Option 2 - Git clone.....	3
2.	Main Vs master branch.....	4
3.	Branch-Rebase-PullRequest-Merge and other Git commands.....	5

1. How do I get my code on Github

How do I get the local Git repo to GitHub. How do I connect them and How do I setup GitHub repo and what do I do on my machine to tell about GitHub repo. There are 2 different approaches to do this.

Option 1: Existing Repo

If you already have an existing repo locally that you want to get on GitHub.

- Create a new repo on GitHub.
- Connect your local repo (add a remote)
- Push up your changes to GitHub.

Option 2: Start From Scratch

If you haven't begun work on your local repo, you can...

- Create a brand-new repo on GitHub.
- Clone it down to your machine.
- Do some work locally.
- Push up your changes to GitHub.

Pushing: To get your own changes and Git history up on GitHub, we need to PUSH them up. The typical workflow looks something like this:

- Make some changes locally.
- Add and commit to those changes.
- Repeat...
- Push new commits up to GitHub.

Option 1

<u>Option 1</u> Move the local Git repo (ex:Website) to empty GitHub branch. Perform <code>\$ git log</code> lot of commits get displayed. All these history and code/work will be moved to GitHub.	
Step 1: Create a new repo on GitHub	1. Sign in on GitHub. 2. In the upper-right corner of page next to profile picture, click “+ - > New Repository.” 3. In new page, enter the <u>Repository</u> name under “Create a new repository.” 4. Enter “Description” 5. Mark it as public or private. If it’s your website, mark it as private, otherwise anyone can make changes to the website. Now we have done empty repository on GitHub. Local repo doesn’t know anything about it.
Step 2: Give a name to GitHub url and set that name in the local Git repo.	Before we can push anything up to GitHub, we need to tell Git about our remote repository on GitHub. We need to set up a "destination" to push up to. In Git, we refer to these "destinations" as remotes. Each remote is simply a URL where a hosted repository lives. To view any existing remotes for you repository, we can run <code>git remote</code> or <code>git remote -v</code> (verbose, for more info) <u>\$ git remote -V</u> This just displays a list of remotes. Since we have not yet set the remote, this command will not show anything. Adding A New Remote: A remote is really two things: a URL and a label. To add a new remote, we need to provide both to Git. <u>\$ git remote add <name> <url></u> Ex: <code>\$ git remote add origin www.bhal.com</code> //Tells Git anytime name "origin" is used, it’s referring to this particular GitHub repo URL. Remember, by setting up a remote we are just telling Git about a remote repository URL. We have not "communicated" with the GitHub repo at all yet.
They are not commonly used, but there are commands to rename and delete remotes if needed. <code>git remote rename <old> <new></code> <code>git remote remove <name></code>	
Step 3: Push changes to remote GitHub	Now that we have a remote set up, let's push some work up to GitHub! To do this, we need to use the <code>git push</code> command. We need to specify the remote we want to push up to AND the specific local branch we want to push up to that remote. <u>\$ git push <remote> <branch></u> Ex: <u>\$ git push origin master</u> //tells git to push up the local-master branch to our origin remote. For the first time, there are no branches on remote. It creates a master branch. From next time, this command pushes local-branch to remote-master by default. But if we don’t want to push to remote-master <code>\$ git push origin <local-branch>: <remote-branch></code> <code>\$ git switch -c cats</code> <code>\$ git push origin cats:master</code> // This pushed cats to master branch
-u option allows us to set the upstream of the branch we are pushing. You can think of this as a link	

connecting our local branch to a branch on GitHub. Running **\$ git push -u origin master** sets the upstream of the local master branch so that it tracks the master branch on the origin repo. Local Git remembers it for future push commands.

Now that it remembers, do some changes and commit.

```
$ git switch -c dogs
```

```
$ git push -u origin dogs
```

```
$ git touch dogs.txt
```

```
$ git commit -m "text push"
```

```
$ git push // changes are pushed to existing dogs branch, not to master.
```

Step 4: Check GitHub repo

Check the recent commits with time reflecting on GitHub. Check "All commits" page.

Option 2 - Git clone

Connecting GitHub repo to local repo

So far, we've created our own Git repositories from scratch, but often we want to get a local copy of an existing repository instead. This opens the path for collaboration and exposure. Or you haven't started your work locally. You know you want to use GitHub. You can start creating empty repo on GitHub and clone it back to local machine. **This automatically configure remote**. Then do some work and push changes to GitHub repo.

To do this, we can clone a remote repository hosted on GitHub. All we need is a URL that we can tell Git to clone for use. To clone a repo, simply run `git clone <url>`. Git will retrieve all the files associated with the repository and will copy them to your local machine. In addition, Git initializes a new repository on your machine, giving you access to the full Git history of the cloned project. Make sure you are not inside of a repo when you clone!. For more information refer : www.git-scm.com/docs/clone

```
$ git clone https://github.com/...
```

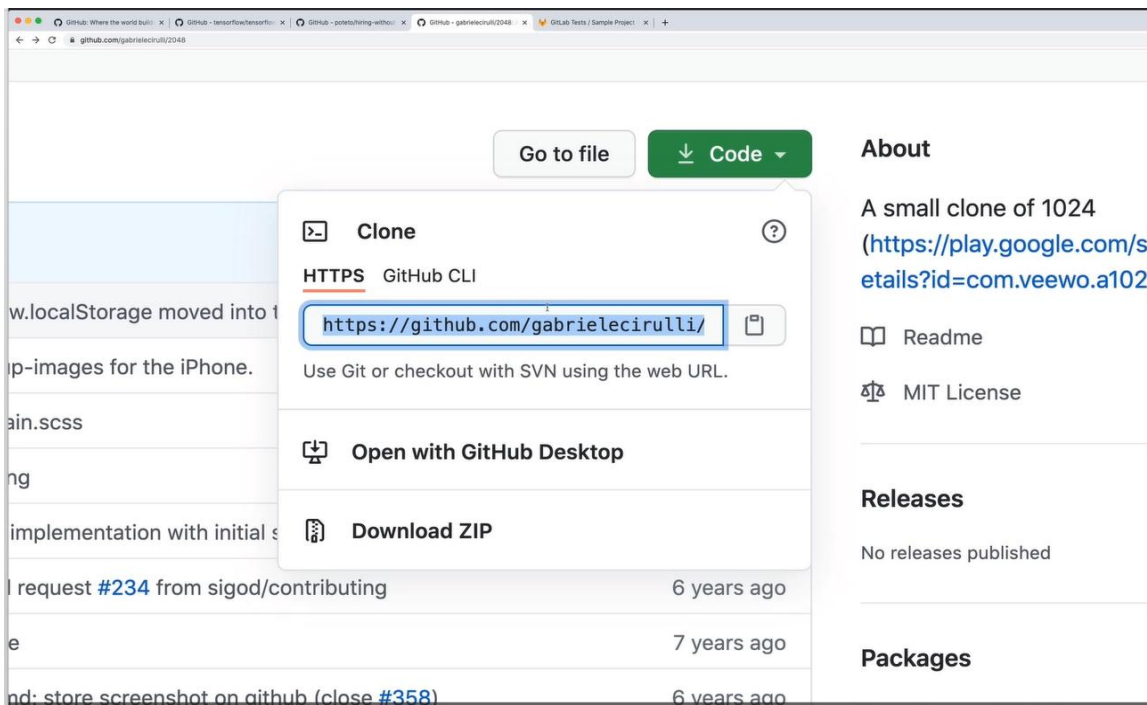
```
// Do some changes.
```

```
$ git diff
```

```
$ git push origin master
```

Anyone can clone a repository from GitHub, **provided the repo is public**. You do not need to be an owner or collaborator to clone the repo locally to your machine. You just need the URL from GitHub. You need permission to push up your own changes to the GitHub repo.

url for cloning can be copied from project path as shown below.



2. Main Vs master branch

In 2020 GitHub started renaming the default branch from master to main. While creating branch new repo on GitHub, if you select “Add a README File”, notice the last line. It says, “main” as default branch. That is instead of creating empty repo, if repo is created with ReadMe file. It automatically wants to add some branch because you cannot commit any file without any branch. We would always change that it always defaults to master in “settings” (lined at the end of the line as shown in picture). Or else go to “settings -> branches -> change default branch “

Owner * Colt / Repository name * colors ✓

Great repository names are short and memorable. Need inspiration? How about **shiny-doodle?**

Description (optional)

☐ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

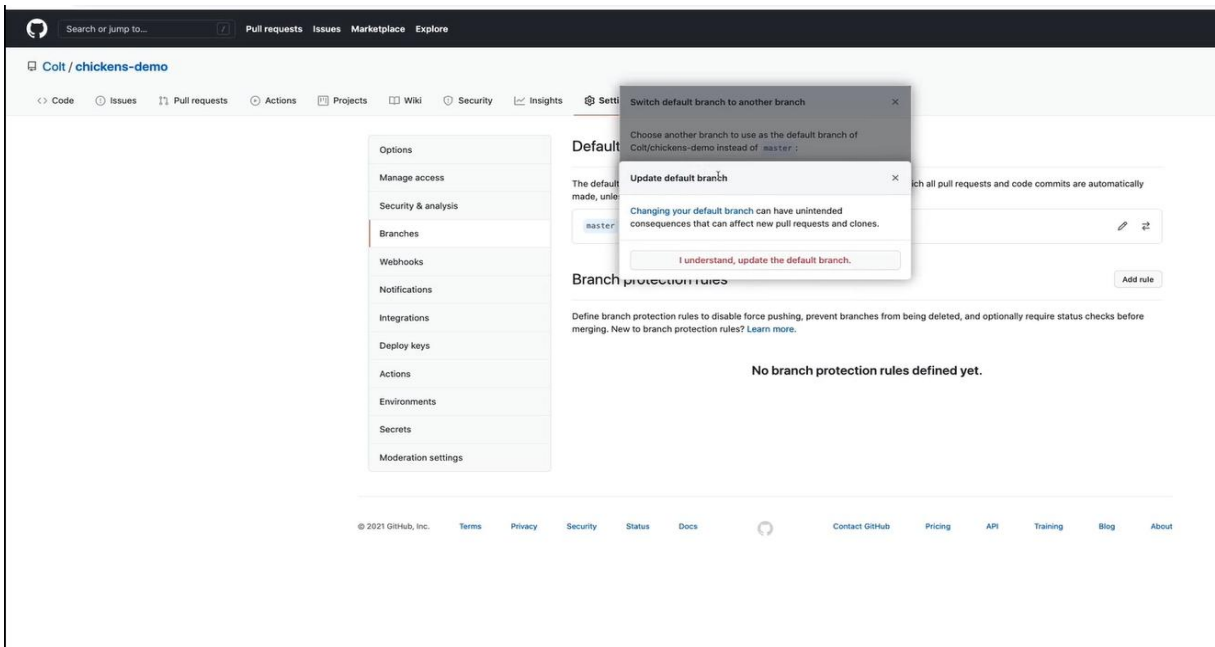
☒ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

☐ **Add .gitignore**
Choose which files not to track from a list of templates. [Learn more.](#)

☐ **Choose a license**
A license tells others what they can and can't do with your code. [Learn more.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

Create repository



You can also rename master to main. There is no significance to branch name.

\$ git branch -M main

3. Branch-Rebase-PullRequest-Merge and other Git commands

Remember GitHub has advantages for collaboration. When more than one person is working on project other commands like Rebase, Merger, pull request come to use.

Collaboration	If you ever plan on working on a project with at least one other person, GitHub will make your life easier! Whether you're building a hobby project with your friend or collaborating with the entire world, GitHub is essential!
Exposure	Your GitHub profile showcases your own projects and contributions to others' projects. It can act as a sort of resumé that many employers will consult in the hiring process. Additionally, you can gain some clout on the platform for creating or contributing to popular projects.
Stay Up to Date	Being active on GitHub is the best way to stay up to date with the projects and tools you rely on. Learn about upcoming changes and the decisions/debate behind them.